

PAPER

Data-driven inverse kinematics for a 6-DOF robotic manipulator using an MPGA-optimized BP neural network

To cite this article: Rong Yao *et al* 2026 *Eng. Res. Express* **8** 055230

View the [article online](#) for updates and enhancements.

You may also like

- [Research on small object detection algorithm for UAV aerial images based on AMF-YOLOv11s](#)
Yaling Chen, Yahong Zhai and Xu Longyan
- [Influence of tool rotational speed on dynamic recrystallization, precipitate evolution, and functional response of friction stir processed magnesium alloy](#)
Raghu Ram N, Sivaji Babu K and B Bala Krishna
- [Research on fault sample generation and decision support system for converter stations combining gan and reinforcement learning](#)
Zhenming Deng, Haichuan Zhang, Yuxuan Zou et al.

Engineering Research Express



PAPER

Data-driven inverse kinematics for a 6-DOF robotic manipulator using an MPGA-optimized BP neural network

Rong Yao^{1,2,*}, Jinsong Wang^{1,2,*}, YuHang Wei^{1,2}, Siqi Huang^{1,2} and Yu Zhao¹

¹ Changchun University of Science and Technology College of Optical and Electrical Engineering, Changchun 130022, People's Republic of China

² Changchun University of Science and Technology Zhongshan Research Institute, Zhongshan 528437, People's Republic of China

* Author to whom any correspondence should be addressed.

E-mail: jsw@mails.cust.edu.cn

Keywords: inverse kinematics, multi-population genetic algorithm, BP neural network, data-driven optimization, 6-DOF robotic manipulator

Abstract

Inverse kinematics (IK) of multi-degree-of-freedom robotic manipulators is challenging due to strong nonlinear coupling, multiple solution branches, and sensitivity to singular configurations. This paper presents a data-driven IK method that integrates a multi-population genetic algorithm (MPGA) with a backpropagation (BP) neural network to improve mapping accuracy and numerical stability. A large training dataset of pose–joint pairs is generated via forward kinematics and Monte Carlo sampling. MPGA performs global optimization of network parameters through Gray-code encoding, adaptive operators, and elitist migration, while the BP model is enhanced with LeakyReLU activation, batch normalization, dropout, and Adam optimization to improve nonlinear fitting capability and regularization performance. Simulation studies on a 6-DOF RM65-B robotic manipulator implemented in MATLAB demonstrate that the optimized model achieves an RMSE of 7.9×10^{-7} , with mean position and orientation errors of 2.4×10^{-4} mm and 1.2×10^{-4} rad, respectively. These errors reflect interpolation accuracy within the sampled workspace rather than general IK performance under uncertainty or model mismatch. Comparative experiments demonstrate superior convergence rates and numerical stability in the simulated environment. These results demonstrate the effectiveness of the proposed MPGA-BP framework for IK solving under offline simulation conditions.

1. Introduction

Robotic manipulators play an essential role in intelligent manufacturing, aerospace operations, medical assistance, and service robotics. A central challenge in these applications is inverse kinematics (IK)—the process of determining joint variables from a desired end-effector pose—which directly affects motion accuracy and trajectory planning [1–3].

Conventional IK methods are broadly classified into analytical and numerical approaches. Analytical solutions, such as those of Paul and others, provide high computational efficiency and precision through closed-form geometric modeling [4, 5]. However, these techniques typically require the manipulator to satisfy specific structural constraints (e.g., Pieper's criterion), which limits their applicability to complex or redundant configurations [6–8]. Numerical approaches, including Jacobian-based iterations and damped least-squares optimization are more general but often exhibit slow convergence, sensitivity to initial conditions, and deterioration near singularities [9–12].

To address these limitations, researchers have increasingly investigated intelligent optimization and learning-based methods. Andrieiev *et al* applied AI-driven control strategies to anthropomorphic robots, but their approach showed limited numerical stability when multiple IK solutions existed [13]. Gao proposed a neural network-based solver that suffered from slow convergence and high sensitivity to initial weights [14]. Luo *et al*

Table 1. Comparison with representative IK methods.

References	Year	Core method	Robot/DOF	Evaluation setting	Main limitations	How this work differs
[14]	2020	BP neural network	6-DOF	Simulation	Initialization-sensitive	Global MPGA initialization
[15]	2022	Multi-objective PSO	6-DOF	Simulation	Computationally expensive	Efficient NN inference
[19]	2024	Improved ABC	Redundant manipulators	Simulation	No learning-based model	Neural IK mapping
[21]	2017	MPGA-based	General robot	Simulation	Parameter-sensitive	Hybrid MPGA–Adam
Proposed method	—	MPGA + BP + Adam	6-DOF	Simulation	Idealized simulation	Scope explicitly stated (simulation-only interpolation)

used a multi-objective particle swarm optimization (PSO) strategy for 6-DOF manipulators, but the method incurred high computational cost and limited generalization ability [15]. Yang *et al* developed a neural model for a 7-DOF arm; however, its performance declined outside the trained workspace [16]. Genetic algorithm (GA)-based approaches, such as those of Zhen *et al*, improved global search capability but still tended to prematurely converge [17], whereas Han *et al*'s welding-oriented IK method lacked broad applicability [18]. Although Shi *et al* enhanced search diversity using an improved artificial bee colony algorithm, achieving fine precision remained difficult [19]. Li *et al* showed that backpropagation (BP) neural networks possess strong nonlinear approximation capability for IK tasks, yet the training process was slow and easily trapped in local minima [20]. Lin *et al* introduced the multi-population genetic algorithm (MPGA) to enhance global exploration, but its performance was highly sensitive to parameter tuning and computationally expensive, restricting real-time implementation [21]. Table 1 summarizes representative IK methods published in recent years and highlights the differences between existing approaches and the proposed MPGA-BP framework.

Overall, most single-algorithm approaches struggle to simultaneously satisfy global optimality, convergence speed, numerical stability, and generalization. This challenge is particularly evident in high-DOF IK problems with highly nonlinear mappings and multiple solution branches. Modern gradient-based optimizers such as Adam provide efficient local convergence. However, they remain sensitive to initial parameter values, which may lead to suboptimal solutions in complex inverse kinematics landscapes. To address these challenges, this paper proposes a hybrid IK methodology that integrates the MPGA with a BP neural network. In the proposed framework, MPGA is not used as a replacement for gradient-based learning. Instead, it serves as a global initialization strategy that optimizes the initial weights and biases of the BP network, thereby reducing the risk of premature convergence. The BP network then performs efficient nonlinear mapping from end-effector pose to joint configuration using gradient-based training. This hybrid design leverages the complementary strengths of global evolutionary search and local gradient optimization, thereby improving accuracy and optimization stability under the simulated data distribution.

Although genetic-algorithm-based neural network approaches for inverse kinematics have been widely reported, most existing studies focus on either single-population optimization or limited integration between evolutionary search and gradient-based learning.

The main contribution of this work lies in the systematic integration of an MPGA with a modern BP neural network architecture. This integration includes Gray-code encoding, adaptive migration, elitist preservation, LeakyReLU activation, batch normalization, dropout regularization, and Bayesian hyperparameter tuning.

Rather than proposing a new standalone optimizer, this study demonstrates how MPGA can be effectively combined with gradient-based training. This combination improves initialization quality, numerical stability, and interpolation accuracy of IK mappings under controlled simulation conditions.

The remainder of this paper is organized as follows. Section 2 presents the kinematic modeling of the RM65-B robotic manipulator. Section 3 describes the proposed MPGA-BP hybrid IK method. Section 4 provides simulation results and comparative analyses. Section 5 concludes the paper and outlines future research directions.

2. Kinematic analysis of the RM65-B robotic manipulator

2.1. Kinematic modeling

The kinematic structure of the RM65-B six-degree-of-freedom robotic manipulator is described using the Denavit–Hartenberg (D–H) convention, which is widely adopted for serial-link robot modeling. Depending on how the coordinate origins are assigned, D–H modeling can be categorized into two variants. One locates the reference frame at the proximal joint, while the other places it at the distal end of each link. Although the two approaches yield different intermediate transformation matrices, they are mathematically equivalent when the base and end-effector reference frames are consistently defined [22]. In this work, the standard D–H formulation proposed by Roth *et al* is employed.

The RM65-B robotic manipulator, which supports a maximum payload of 5 kg, serves as the experimental platform in this study. Its forward kinematic model is developed by incorporating both structural geometry and joint motion constraints. Figure 1 illustrates the coordinate frames assigned to each joint based on the D–H convention, while table 2 lists the joint rotation limits and maximum angular velocities.

To ensure consistent geometric representation and simplify spatial transformations, the second variant of the D–H formulation is adopted in this study, where each coordinate frame is placed at the distal end of its corresponding link. This approach yields a uniform parameter description along the kinematic chain and provides a clear basis for subsequent inverse kinematics derivations and optimization.

Table 3 summarizes the complete D–H parameter set of the RM65-B manipulator, where θ_i , d_i , a_i and α_i represent the joint rotation angle, link offset, link length, and link twist angle of the i th joint, respectively.

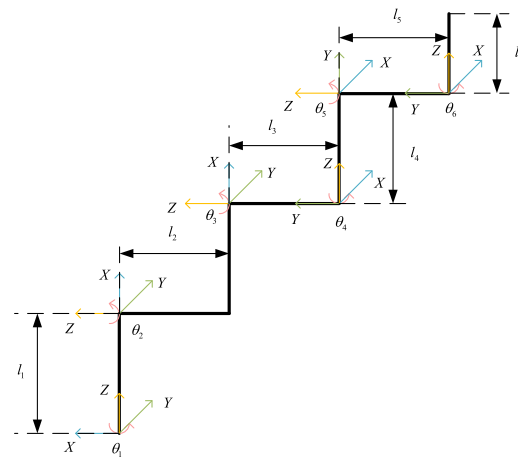


Figure 1. Schematic diagram of the reference coordinate system of the RM65-B robotic arm.

Table 2. Technical parameters of RM65-B robotic arm.

Joint	Joint rotation range/(°)	Maximum speed/[(°) · s ⁻¹]
1	−178 ~ 178	180
2	−130 ~ 130	180
3	−135 ~ 135	225
4	−178 ~ 178	225
5	−128 ~ 128	225
6	−360 ~ 360	225

Table 3. RM65-B robotic arm D–H parameter table.

Joint	θ_i /(°)	d_i /mm	a_i /mm	α_i /(°)
1	θ_1	240.5	0	0
2	θ_2	0	152.5	−90
3	θ_3	0	210	0
4	θ_4	0	81.5	−90
5	θ_5	115	0	90
6	θ_6	0	0	0

The homogeneous transformation matrix from link i to link $i + 1$ is defined by equation (1)[23].

$${}^{i-1}_iT = \begin{bmatrix} c\theta_i & -s\theta_i c\alpha_i & s\theta_i s\alpha_i & a_i c\theta_i \\ s\theta_i & c\theta_i c\alpha_i & -c\theta_i s\alpha_i & a_i s\theta_i \\ 0 & s\alpha_i & c\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

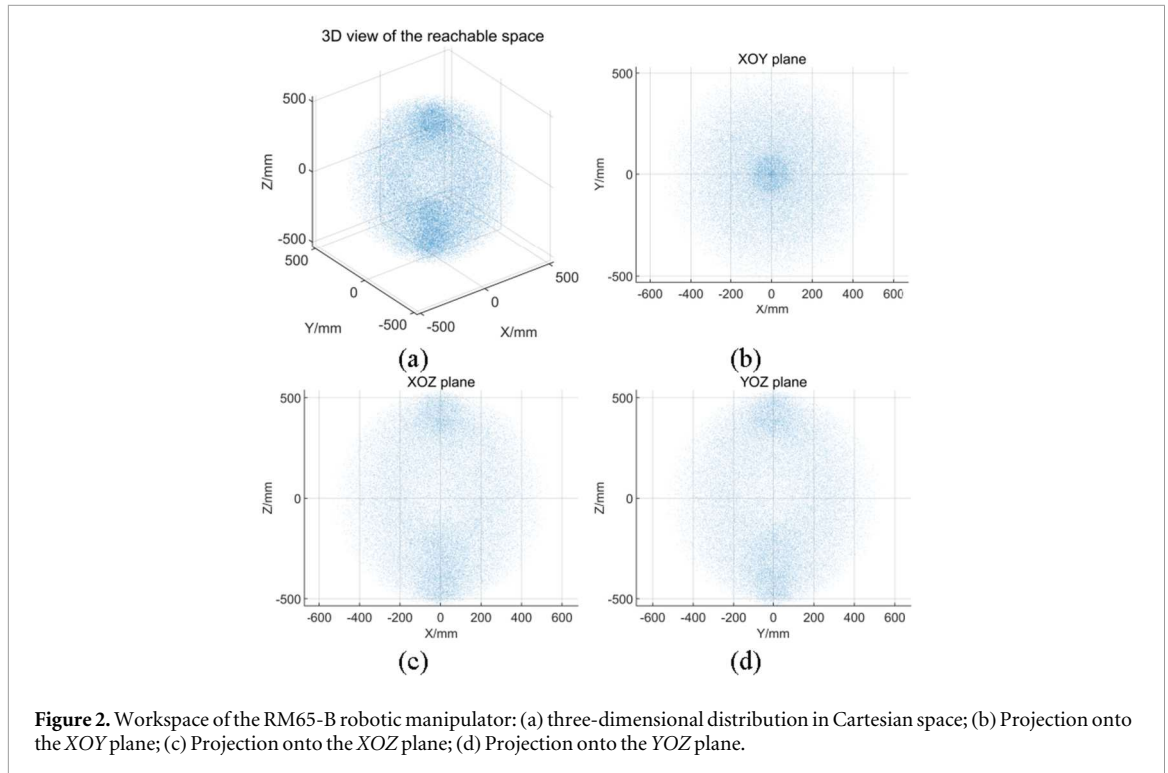
where a_i , α_i , d_i are constants determined by the manipulator geometry, θ_i is the joint variable, and c and s denote cosine and sine terms, respectively.

The overall forward kinematic transformation from the base frame to the end-effector is obtained by sequential multiplication of the six individual link transformations, as expressed in equation (2).

$${}^0_6T = {}^0_1T {}^1_2T {}^2_3T {}^3_4T {}^4_5T {}^5_6T \quad (2)$$

yielding the rotation matrix $R(n_x, n_y, n_z, o_x, o_y, o_z, a_x, a_y, a_z)$ and position vector $P(p_x, p_y, p_z)$, as shown in equation (3).

$${}^0_6T = \begin{bmatrix} r_{11} & r_{12} & r_{13} & p_x \\ r_{21} & r_{22} & r_{23} & p_y \\ r_{31} & r_{32} & r_{33} & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} n_x & o_x & a_x & p_x \\ n_y & o_y & a_y & p_y \\ n_z & o_z & a_z & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3)$$



The above formulation enables direct determination of the end-effector pose from known joint parameters, constituting the forward kinematics of the RM65-B manipulator. Conversely, the inverse kinematics problem aims to determine the joint angles corresponding to a desired end-effector pose, which motivates the learning-based optimization approach developed in this study.

2.2. Workspace analysis of the RM65-B manipulator

The workspace of a robotic manipulator is a crucial metric that reflects its structural characteristics and motion capability. Based on the joint rotation limits of the RM65-B robotic arm listed in table 2, the Monte Carlo (MC) sampling method is employed to generate a large number of random joint configurations. This process approximates the reachable region of the end-effector in Cartesian space. Specifically, each joint angle θ is sampled uniformly within its allowable range $[\theta_{\min}, \theta_{\max}]$, ensuring unbiased exploration of the workspace. The sampled joint angles are generated using equation (4).

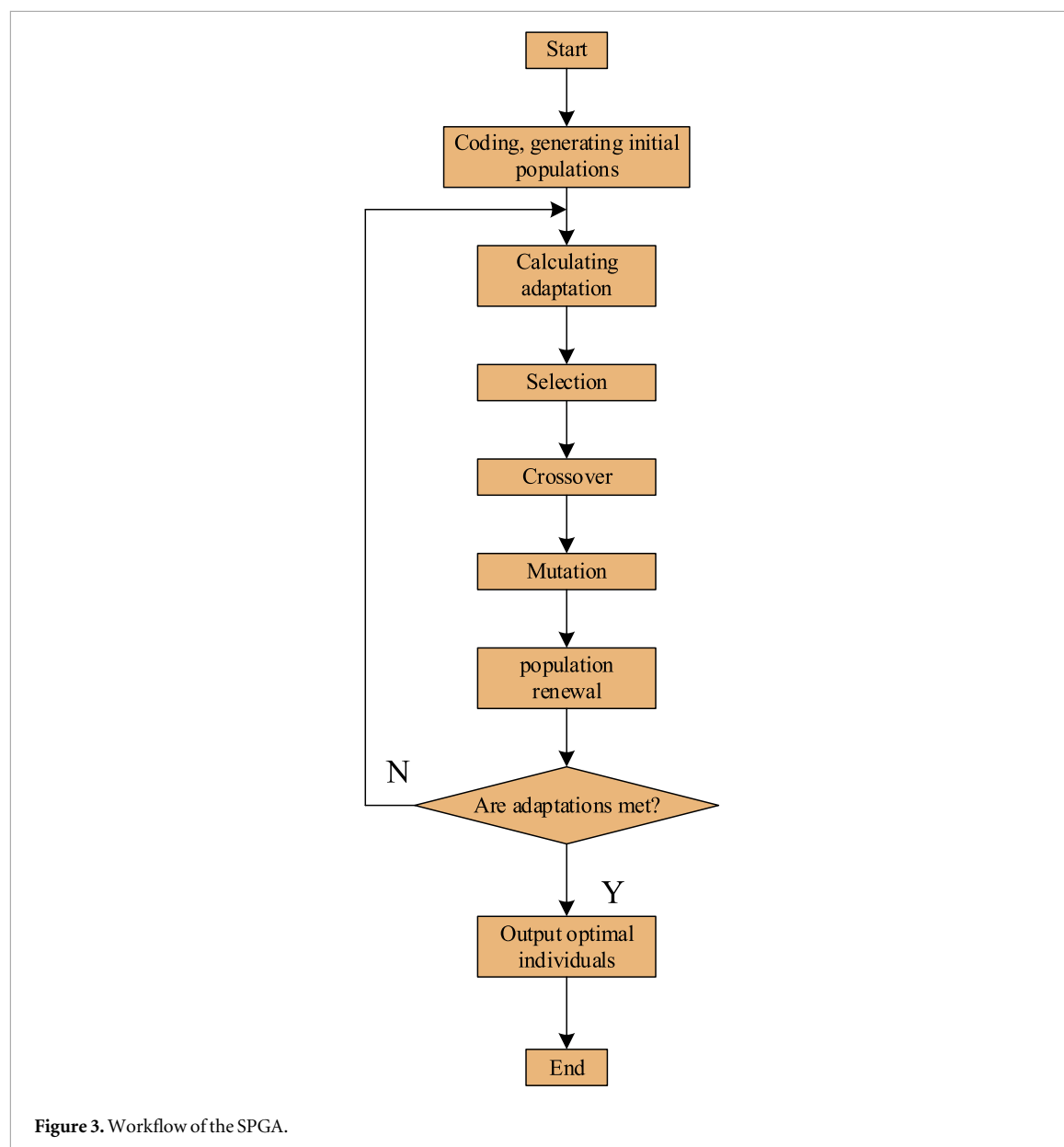
$$\theta = (\theta_{\max} - \theta_{\min})\text{Rand}(N, 1) + \theta_{\min} \quad (4)$$

where $\text{Rand}(N, 1)$ denotes a sequence of uniformly distributed random variables, and N is the total number of sampled configurations. Each sampled joint-angle vector is subsequently substituted into the forward kinematics model to compute the corresponding end-effector position. By aggregating these positions, the reachable workspace of the manipulator in three-dimensional space can be effectively approximated.

Figure 2(a) illustrates the spatial distribution of $N = 50,000$ randomly generated end-effector poses. As shown, the reachable region can be approximated geometrically by a sphere with a diameter of approximately 539 mm. Figures 2(b)–(d) present the projections of the workspace onto the XOY, XOZ, and YOZ planes, respectively. The projections reveal a uniform spatial distribution with strong internal continuity, indicating that the manipulator is capable of accessing points across the entire spherical domain.

These results demonstrate that the RM65-B robotic arm possesses a compact and well-defined workspace, enabling effective reachability in all principal directions. This confirms the manipulator's strong spatial coverage, structural flexibility, and operational capability.

The Monte Carlo sampling process provides a reliable and comprehensive representation of the reachable domain, forming the basis for subsequent IK performance evaluation.



3. MPGA–BP IK solution modeling

3.1. Single-population genetic algorithm (SPGA)

The single-population genetic algorithm (SPGA) is a classical optimization method inspired by natural selection and evolutionary genetics. It searches for optimal solutions by simulating biological evolution, in which a population of candidate solutions evolves toward higher fitness over successive generations.

In SPGA, candidate solutions are randomly initialized to define the search space. Each solution is evaluated using a fitness function, and those with higher fitness are preferentially selected. In each generation, the population undergoes selection, crossover, and mutation. This evolutionary process gradually improves the population fitness and guides the search toward the optimal solution.

Due to its strong global search capability, SPGA has been applied in structural optimization, route planning, neural network parameter tuning, and multi-objective scheduling. The overall workflow of SPGA is shown in figure 3.

The crossover probability P_c and mutation probability P_m significantly affect the global search capability and convergence behavior of SPGA [24]. A very small P_c restricts the search space and slows convergence, whereas an excessively large P_c may cause oscillation and instability. Similarly, a small P_m increases the risk of local optima, while a large P_m may disrupt the inheritance of high-quality genetic information [25].

To mitigate premature convergence and improve search efficiency, SPGA generally adopts adaptive strategies that dynamically adjust P_c and P_m during the evolution. The adaptive update rules for P_c and P_m are given in equations (5)–(7).

$$P_c = \begin{cases} 0.1, & i = 1 \\ 1/(1 + e^{(-1 \times af / (|bf(i) - wf(i)|)})}, & i > 1 \end{cases} \quad (5)$$

$$af = \sum_{i=1}^{num} (f(i) - bf(i)) / gn \quad (6)$$

$$P_m = \begin{cases} 0.01, & i = 1 \\ 1/(1 + e^{(-0.1 \times af / (|bf(i) - wf(i)|)})} - 0.5, & i > 1 \end{cases} \quad (7)$$

In equation (7), af denotes the average fitness difference of the population, which reflects the overall evolutionary progress.

In high-dimensional optimization problems, SPGA suffers from increased coding complexity, higher computational burden, and reduced stability. Moreover, SPGA cannot efficiently exploit parallel computing. These limitations reduce its practicality for applications requiring high accuracy and stability, such as robotic motion control.

3.2. Multi-population genetic algorithm (MPGA)

Compared with the traditional SPGA, the MPGA introduces three major enhancements:

- (1) Parallel evolution among multiple subpopulations. Each subpopulation evolves independently in different regions of the search space, preserving population diversity and reducing the risk of premature convergence.
- (2) Inter-population migration. A migration operator periodically exchanges individuals between subpopulations, encouraging cooperative search behavior and preventing stagnation in suboptimal regions.
- (3) Elitist selection mechanism. The best individuals of each subpopulation are preserved in every generation, and these elite candidates are utilized to evaluate convergence quality.

The workflow of MPGA is illustrated in figure 4.

In figure 4, g denotes the generation index; M denotes the number of individuals per subpopulation; N denotes the total number of subpopulations; and G denotes the maximum number of generations. The fitness value of the g th generation across all subpopulations is represented as $Fitness(P(g, N)M)$, while $P(g, n)$ denotes the n th subpopulation in the g th generation.

Through parallel evolutionary search, inter-population migration, and elite retention, MPGA enhances population diversity and accelerates convergence. These mechanisms improve convergence stability and help overcome common SPGA limitations, such as premature convergence in high-dimensional search spaces.

3.3. BP neural network

A BP neural network uses the error backpropagation algorithm as its core learning mechanism, iteratively adjusting network weights and biases to achieve nonlinear mapping between inputs and outputs. Due to its strong approximation capability, BP has been widely employed in prediction, pattern recognition, and nonlinear regression tasks.

In this study, the input vector of the BP neural network is defined as shown in equation (8).

$$x = [p_x, p_y, p_z, r_{11}, r_{12}, \dots, r_{33}]^T \quad (8)$$

where p_x, p_y, p_z denote the end-effector position, and r_{ij} are the elements of the rotation matrix.

In this study, the end-effector orientation is represented using the full rotation matrix $R \in \mathbb{R}^{3 \times 3}$ rather than Euler angles or quaternions. Although this representation is redundant (9 parameters for 3 degrees of freedom), it avoids singularities and discontinuities associated with Euler angles and removes the unit-norm constraint required by quaternions. The redundancy is handled implicitly by the neural network during training. Since all samples are generated from an ideal forward kinematics model, the rotation matrices are numerically consistent and do not introduce instability. However, using a full rotation matrix may increase input dimensionality and sensitivity to noise, which will be investigated in future work.

The network output vector is defined as shown in equation (9).

$$\theta = [\theta_1 \ \theta_2 \ \theta_3 \ \theta_4 \ \theta_5 \ \theta_6]^T \quad (9)$$

which represents the predicted joint angles of the 6-DOF manipulator.

The BP neural network consists of an input layer, three hidden layers, and an output layer. For a network with L layers, the forward propagation process is expressed as shown in equations (10) and (11).

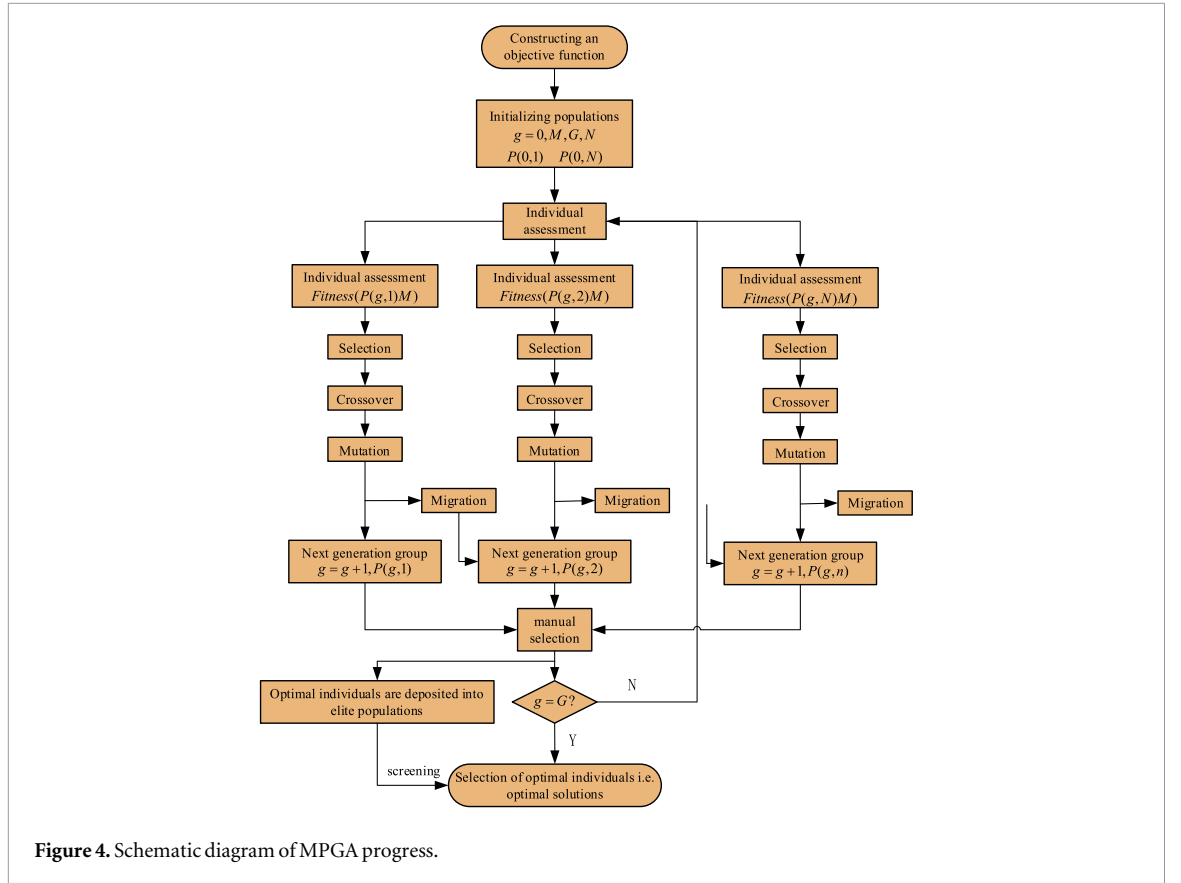


Figure 4. Schematic diagram of MPGA progress.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)} \quad (10)$$

$$a^{(l)} = \varphi^{(l)}(z^{(l)}), \quad l = 1, 2, \dots, L \quad (11)$$

where $W^{(l)}$ and $b^{(l)}$ denote the weight matrix and bias vector of the l th layer, respectively, $a^{(l)}$ is the activation output, and $\varphi^{(l)}(\cdot)$ is the activation function, In this study, $L = 4$, including three hidden layers and one output layer.

For clarity and reproducibility, the dimensions of all weight matrices and bias vectors are specified as shown in equation (12). Let $n_0 = 12$ denote the input dimension, and $n_1 = 512$, $n_2 = 256$, $n_3 = 128$, $n_4 = 6$ denote the numbers of neurons in hidden layers 1–3 and the output layer, respectively. For layer $l = 1, \dots, 4$, the weight matrix and bias vector are defined in equation (12).

$$W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}, \quad b^{(l)} \in \mathbb{R}^{n_l} \quad (12)$$

Accordingly, the specific dimensions in this study are given in equation (13).

$$\begin{aligned} W^{(1)} &\in \mathbb{R}^{512 \times 12}, \quad b^{(1)} \in \mathbb{R}^{512}; \quad W^{(2)} \in \mathbb{R}^{256 \times 512}, \quad b^{(2)} \in \mathbb{R}^{256}; \\ W^{(3)} &\in \mathbb{R}^{128 \times 256}, \quad b^{(3)} \in \mathbb{R}^{128}; \quad W^{(4)} \in \mathbb{R}^{6 \times 128}, \quad b^{(4)} \in \mathbb{R}^6. \end{aligned} \quad (13)$$

In this work, the LeakyReLU activation function used in all hidden layers is defined in equation (14).

$$\phi(z) = \max(\alpha z, z) \quad (14)$$

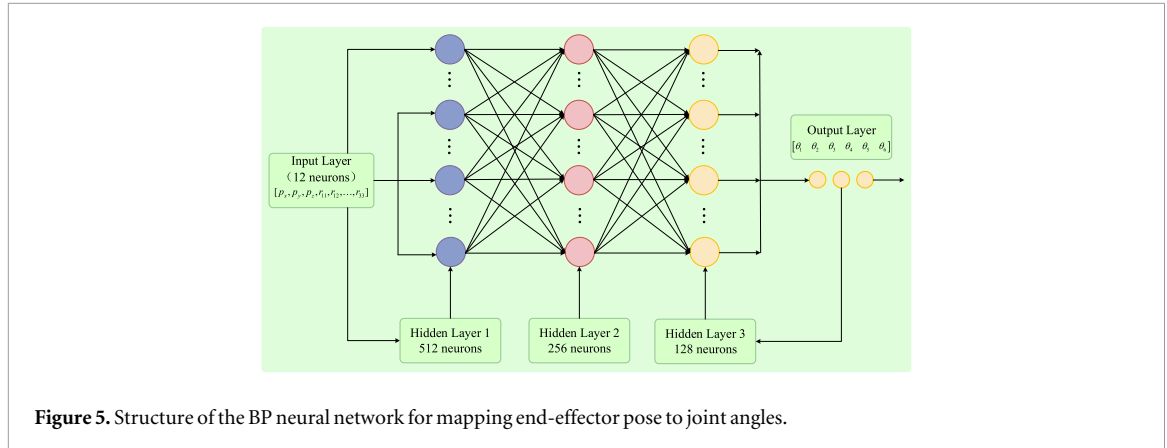
where α is a small positive constant. The output layer adopts a linear activation function to perform regression on joint angles.

The training objective is to minimize the mean squared error (MSE) between the predicted and true joint angles, as defined in equation (15).

$$L = \frac{1}{N} \sum_{i=1}^N \|\theta_i^{\text{pred}} - \theta_i^{\text{true}}\|^2 \quad (15)$$

The network parameters are updated using gradient-based backpropagation, as defined in equations (16)–(17). Specifically, the weight matrices and bias vectors are iteratively updated according to the gradient descent rules:

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}} \quad (16)$$



$$b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial L}{\partial b^{(l)}} \quad (17)$$

where η is the learning rate. In this study, the Adam optimizer is employed to adaptively update the parameters based on first- and second-order moment estimates of the gradients.

The BP neural network can model complex nonlinear relationships through multilayer transformations. In practice, one to three hidden layers are usually sufficient, while deeper networks increase computational cost and convergence difficulty.

As illustrated in figure 5, the BP neural network consists of an input layer with 12 neurons, three hidden layers (512, 256, and 128 neurons), and an output layer with 6 neurons. The inputs represent the end-effector position (p_x, p_y, p_z) and the rotation matrix elements ($r_{11}, r_{12}, \dots, r_{33}$). All hidden layers use the LeakyReLU activation function, while the output layer uses a linear activation function to regress the six joint angles.

3.4. MPGA-based optimized BP neural network

Building upon the BP learning mechanism and the MPGA strategy, an integrated framework is developed to improve IK solution accuracy. In this work, MPGA optimizes only the initial weights and biases of a fixed BP architecture (12–512–256–128–6). The network structure (number of layers and neurons), input–output representation, and activation functions are not optimized by MPGA. After MPGA provides an optimized initialization, the parameters are further updated by gradient-based BP training using the Adam optimizer. The complete optimization procedure is illustrated in figure 6.

The detailed procedure of applying MPGA to optimize the BP neural network is summarized as follows:

- (1) **Data Generation and Preprocessing.** The training dataset is constructed using the forward kinematic model of the robotic arm. A large number of random joint configurations are generated, and the corresponding end-effector poses (position and orientation) are calculated. Each sample consists of the desired end-effector pose as network input and the associated joint angles as network output. Prior to training, input and output data are normalized and partitioned into training, validation, and test sets to ensure efficient learning and reliable evaluation within the same simulated data distribution.
- (2) **BP Network Architecture and Training Strategy.** The BP neural network consists of an input layer, three hidden layers, and an output layer. The input layer accepts 12 features, including the end-effector position (p_x, p_y, p_z) and the elements of the rotation matrix R (see section 3.3). The hidden layers contain 512, 256, and 128 neurons respectively. To enhance generalization and reduce overfitting, dropout and batch normalization are incorporated into the network architecture. A LeakyReLU activation function is adopted to improve nonlinear mapping capability. The output layer performs regression to predict the six joint angles ($\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6$). The model is trained using the Adam optimizer with L_2 regularization. A dynamic learning-rate schedule decreases the rate every 20 epochs, while early-stopping based on validation performance prevents overfitting. Bayesian optimization is employed to automatically tune key training hyperparameters—such as the learning rate, L_2 regularization coefficient, and dropout rate—using the RMSE of the validation set as the optimization criterion. The network architecture (12–512–256–128–6) is fixed throughout the study.
- (3) **Population Initialization and Gray Code Encoding.** The genetic optimization process begins by initializing multiple subpopulations in order to enhance search efficiency and improve continuity among neighboring candidate solutions. Within the MPGA framework, each individual encodes a candidate parameter set of

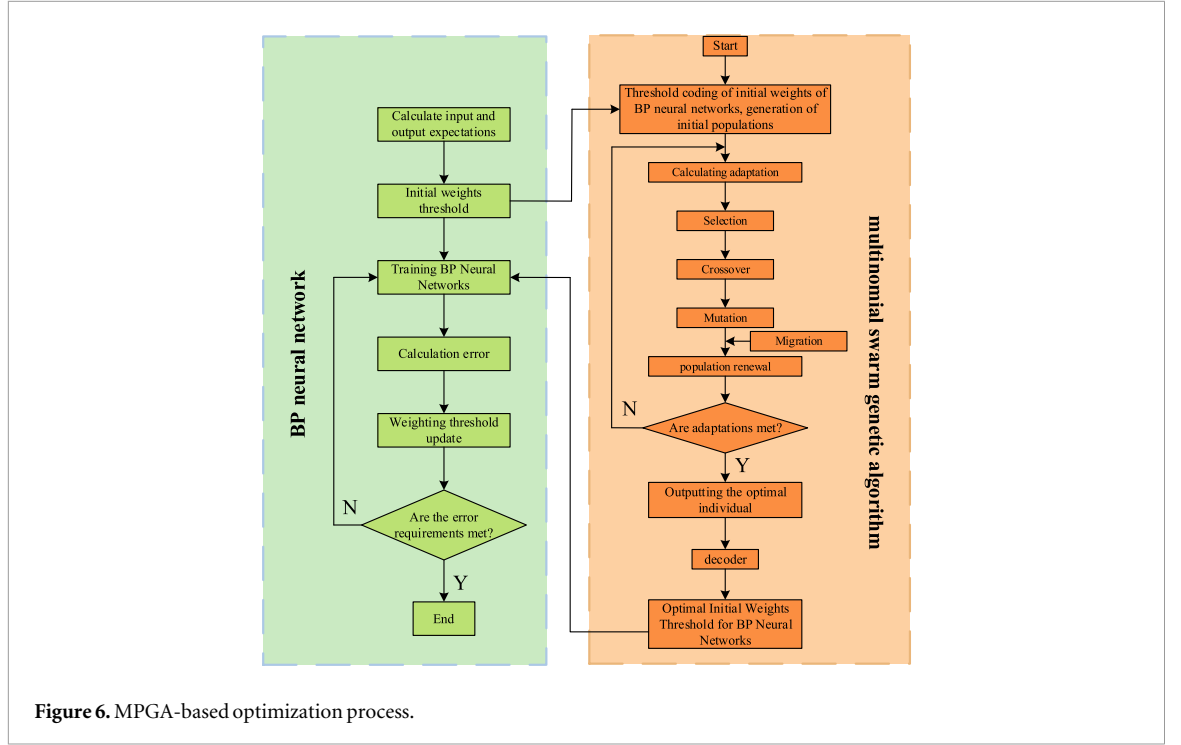


Figure 6. MPGA-based optimization process.

the BP neural network. The total population size is denoted by P_s , the number of individuals per subpopulation by N_s , and the maximum number of evolutionary generations by G , all of which are predefined prior to optimization. To strengthen search stability, the weights and biases of the BP network are encoded using Gray code encoding. Compared with standard binary encoding, Gray encoding effectively reduces the Hamming distance between adjacent individuals, thereby improving local exploration capability and accelerating convergence. Each encoded individual is represented as a binary string, which is decoded into real-valued weights and biases during fitness evaluation. The conversion from binary code $a_1 a_2 \dots a_n$ to Gray code $b_1 b_2 \dots b_n$ follows equation (18).

$$b_i = \begin{cases} a_i & i = 1 \\ \text{mod}(a_i + a_{i-1}, 2) & i > 1 \end{cases} \quad (18)$$

To reduce computational overhead, a three-dimensional matrix is used to store the encoded population data, denoted (x, y, z) , where x represents the number of individuals in each population, y denotes the length of the binary chromosome, and z indicates the population index.

The crossover and mutation probabilities, P_c and P_m , are dynamically updated according to equations (19)–(20).

$$P_c = 0.7 + (0.9 - 0.7) \times \text{rand}(N, 1) \quad (19)$$

$$P_m = 0.001 + (0.05 - 0.001) \times \text{rand}(N, 1) \quad (20)$$

Typically, the crossover probability P_c is selected within the range of 0.7–0.9, while the mutation probability P_m is set within the range of 0.001–0.05 to maintain balance between exploitation and exploration.

Let $\Theta = \{W^{(l)}, b^{(l)}\}_{l=1}^4$ denote the set of trainable parameters of the BP network, where $W^{(l)}$ and $b^{(l)}$ are defined in section 3.3. MPGA searches for an optimized initialization $\Theta^{(0)}$ by minimizing the validation loss in equation (21).

$$\Theta_{\text{MPGA}}^{(0)} = \arg \min_{\Theta^{(0)} \in \Omega} \mathcal{L}_{\text{val}}(\Theta^{(0)}) \quad (21)$$

where Ω is the feasible range of encoded weights and biases (bounded parameter space). After MPGA-based initialization, BP training updates the parameters iteratively as in equation (22).

$$\Theta^{(t+1)} = \Theta^{(t)} - \eta \nabla_{\Theta} \mathcal{L}_{\text{train}}(\Theta^{(t)}), \quad \Theta^{(0)} = \Theta_{\text{MPGA}}^{(0)} \quad (22)$$

In this work, the gradient-based update is implemented using the Adam optimizer.

- (4) **Fitness Function Evaluation.** The fitness value of each individual is computed by quantifying the discrepancy between the predicted end-effector pose and the desired pose. The objective function is defined in equation (23).

$$F(x) = P_{cur} - P_{des} = \begin{pmatrix} \Delta R & \Delta p \\ 0 & 1 \end{pmatrix} \quad (23)$$

The pose error consists of two components:

- (1) The orientation error matrix ΔR , and
- (2) The position error vector Δp .

Specifically, ΔR represents the deviation between the predicted and target orientation matrices, while Δp denotes the Euclidean positional difference between the predicted and target end-effector positions.

To explicitly model these aspects, the objective function $F(x)$ is expressed as a system of nonlinear equations in equation (24).

$$F(x) = \begin{cases} f_1(x) = \|\Delta p\| = \|p_c - p_d\| \\ f_2(x) = \|\Delta R\|_F = \|R_c - R_d\|_F \end{cases} \quad (24)$$

where $\|\cdot\|$ denotes the Euclidean norm for position error, and $\|\cdot\|_F$ denotes the Frobenius norm for orientation error magnitude.

To integrate the position and orientation errors into a unified metric suitable for evolutionary optimization, the combined fitness function is formulated in equation (25).

$$Fitness = \frac{1}{1 + \omega_o \|\Delta R\| + \omega_p \|\Delta p\|} \quad (25)$$

Here, ω_o and ω_p are weighting coefficients that balance the contributions of orientation and position errors. In this study, $\omega_p = 1$ and $\omega_o = 0.001$ are empirically selected to maintain numerical stability and ensure appropriate scaling between the two error terms.

It should be noted that the choice of these weighting coefficients influences the convergence behavior and the relative emphasis between position and orientation accuracy. A systematic sensitivity analysis of ω_o and ω_p is beyond the scope of this work and will be investigated in future studies.

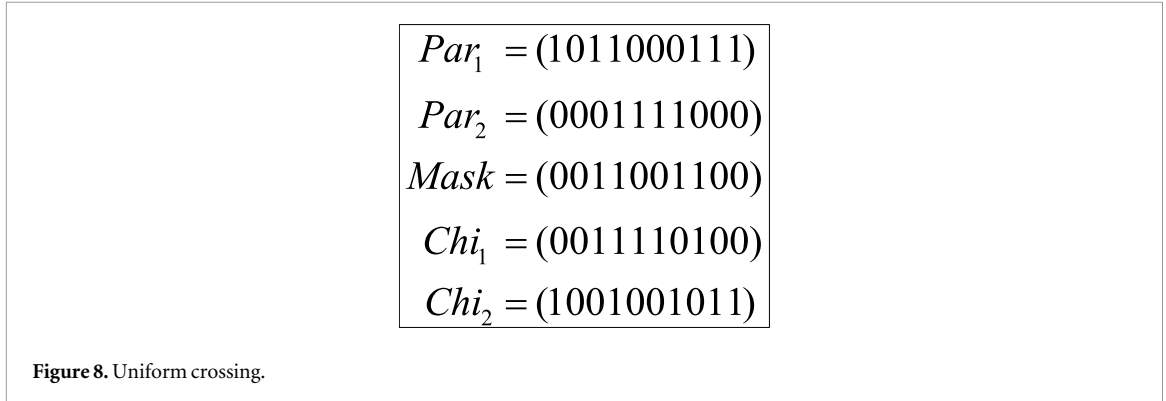
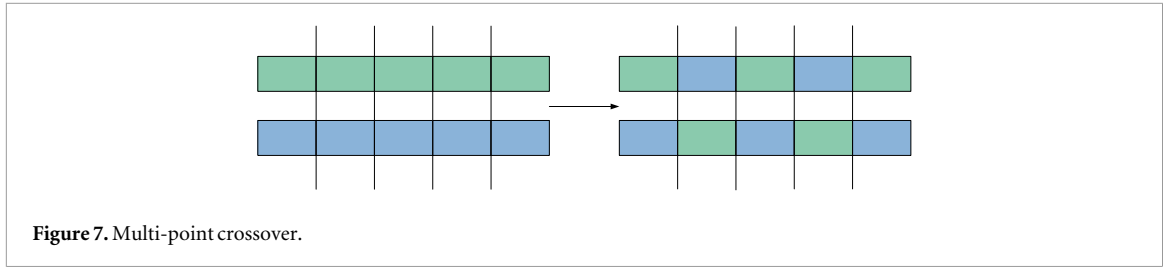
In addition to optimization via MPGA, the predictive performance of the BP neural network is evaluated independently using the RMSE on the training set, as defined in equation (26).

$$RMSE = \sqrt{\frac{1}{M} \sum_{i=1}^M (\hat{q}_i - q_i)^2} \quad (26)$$

where $\hat{q}_i$ is the predicted joint angle, q_i is the true joint angle, and M is the total number of training samples. A lower RMSE indicates improved prediction accuracy and better in-distribution prediction performance of the MPGA-optimized BP neural network.

- (5) **Selection Operation.** The MPGA employs a fitness-proportionate selection strategy, commonly referred to as roulette-wheel selection. Each individual is assigned a selection probability proportional to its fitness, thereby increasing the likelihood that high-fitness individuals contribute their genetic material to the next generation. To further stabilize evolutionary progress, an elitist preservation mechanism is introduced. Specifically, the best-performing individual from each subpopulation is directly retained and subsequently incorporated into the migration operator. This elitist–migration coupling ensures that high-quality solutions propagate across subpopulations, effectively enhancing global exploration while preventing the loss of superior genetic information.
- (6) **Crossover Operation.** To enhance population diversity and suppress premature convergence, a hybrid crossover mechanism integrating multi-point crossover and uniform crossover is adopted. As evolution proceeds, the probability of uniform crossover is gradually increased to strengthen fine-grained genetic mixing.

During the multi-point crossover stage, multiple crossover locations are selected along the chromosome, and the corresponding gene segments between two parent chromosomes are exchanged, as shown in figure 7. This operation redistributes genetic information on a segment-wise basis and significantly improves global exploration capability.



Uniform crossover is further applied to refine genetic recombination at the bit level. A randomly generated binary mask determines the inheritance pattern for each gene bit (figure 8). A mask bit of '1' assigns the gene to the offspring from Par_1 , whereas a mask bit of '0' assigns the gene from Par_2 . The complementary mask produces the second offspring.

This bit-level mechanism facilitates a more uniform dispersion of genes across chromosomes, which provides stronger robustness against local optima.

- (7) Mutation Operation. A dynamic mutation mechanism is adopted to adjust the mutation probability across generations. The mutation rate $P_m(i)$ of the i th subpopulation is adaptively updated according to its crossover rate $P_c(i)$, the current generation index gen , and the total evolutionary length GEN . The parameter M specifies the precision of the mutation adjustment. The update rule is given in equation (27).

$$P_m(i + 1) = P_m(i) + \left(\frac{gen}{GEN} \right)^M \times \left(\frac{P_c(i)}{3} - P_m(i) \right) \quad (27)$$

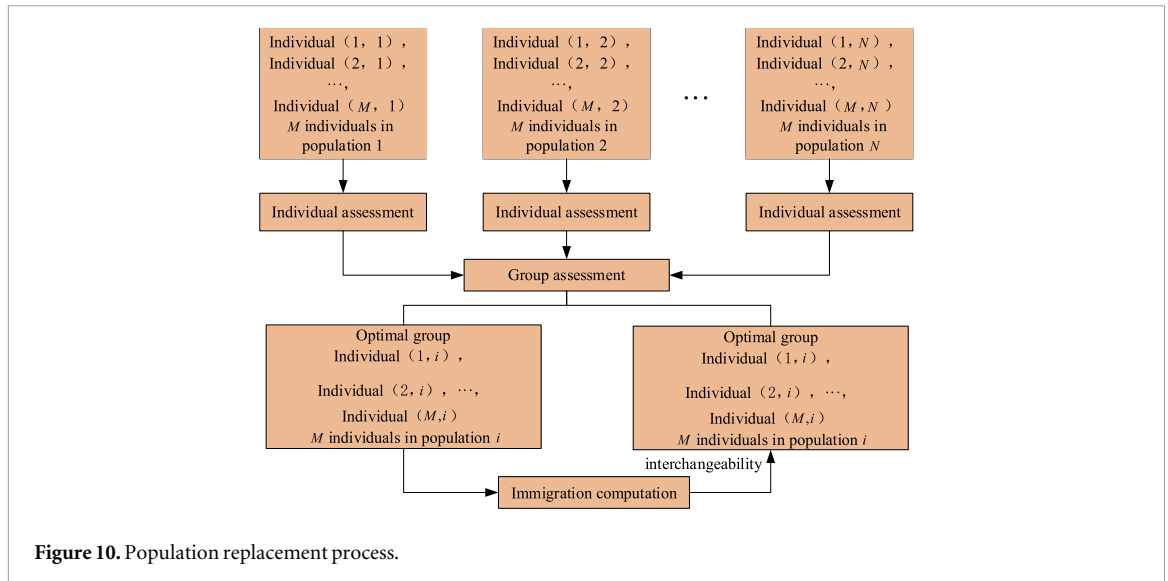
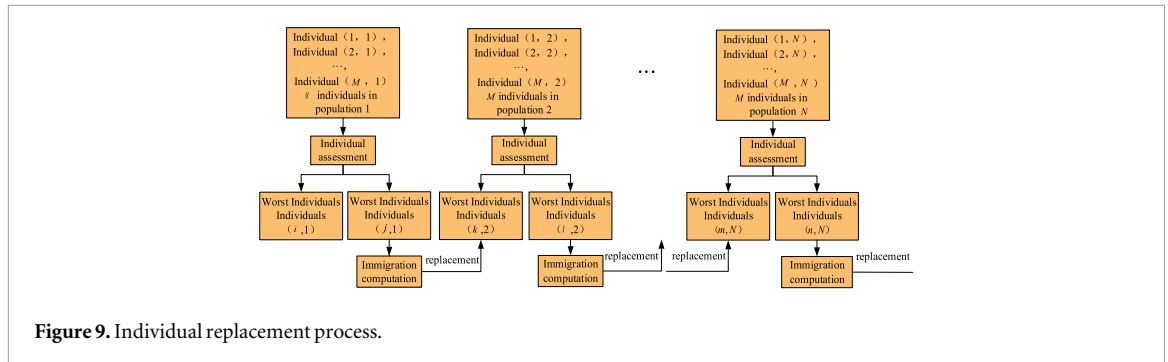
This adaptive strategy gradually increases the mutation probability as evolution progresses, thereby preserving population diversity and effectively preventing premature convergence in the later stages of optimization.

- (8) Migration Operation. Combined Population-Level and Individual-Level Replacement. To further enhance global search diversity and prevent population stagnation, a migration operator is incorporated into the MPGA framework. Since different subpopulations typically explore distinct regions of the search space, migration enables the exchange of high-quality individuals across subpopulations, strengthening information flow and improving overall evolutionary efficiency.

At each generation, an individual-evaluation operator identifies the best and worst individuals within every subpopulation. Following a predefined migration sequence, the worst-performing individuals in a target subpopulation are replaced by the best-performing individuals from the corresponding source subpopulation. This replacement mechanism elevates the fitness level of each subpopulation while preserving search diversity.

To maintain continuity among subpopulations and avoid information loss, a ring-migration strategy is adopted. Specifically, the worst individual in the first subpopulation is replaced by the best individual from the last subpopulation, forming a closed-loop migration cycle. This mechanism ensures that elite individuals are circulated across all subpopulations, as illustrated in figure 9.

Population-level replacement further contributes to evolutionary stability by selectively substituting poorly adapted individuals with newly generated candidates. During the early stages of evolution, a higher rate of population-level replacement is applied to expand solution diversity and accelerate global exploration. As



evolution proceeds, this replacement rate is gradually reduced, allowing individual-level refinement—such as mutation and local search—to dominate. This shift enhances local exploitation and improves convergence precision and stability. The overall population replacement process is depicted in figure 10.

- (9) **Elitist Selection Strategy.** An elitist selection strategy is employed to preserve the best-performing individuals from each subpopulation. These elite individuals are stored within a dedicated elite pool and are excluded from subsequent selection, crossover, and mutation operations. The elite pool is updated periodically to prevent the degradation of high-quality solutions caused by stochastic genetic operators, ensuring stable inheritance of superior genetic traits throughout the evolutionary process.
- (10) **Global Optimal Population Update.** At the end of each generation, the fitness of the entire population is evaluated. If the best-performing individual of the current generation surpasses the previously recorded global optimum, the global optimal solution is updated. This mechanism ensures that the overall solution quality continues to improve as the optimization progresses.
- (11) **Termination Condition.** The evolutionary process terminates when the maximum number of generations is reached. If this termination condition is not met, the algorithm returns to the evolutionary operations (starting from Step 5) to further refine the solutions and continue optimizing the weights and biases of the BP neural network.
- (12) **BP Network Parameter Assignment.** Once MPGA identifies an optimal parameter set, the corresponding encoded individual is decoded into real-valued network parameters, including weights and biases. These parameters are then assigned to the BP neural network, thereby completing the model reconstruction. The optimized network can subsequently be used for forward inference or additional fine-grained training.
- (13) **Convergence Assessment.** The convergence performance of the optimized BP neural network is evaluated using the RMSE computed on the validation dataset. If the prediction error does not meet the predefined convergence threshold, the weighting coefficients for orientation and position errors (i.e., ω_o and ω_p) can

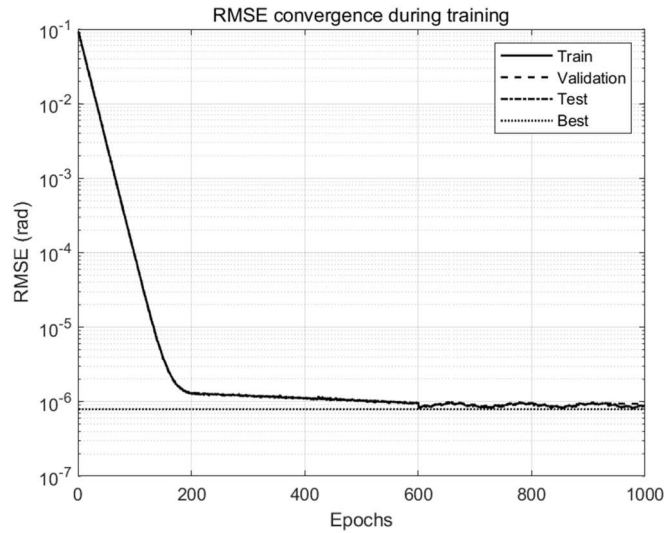


Figure 11. RMSE convergence curve of the MPGA-optimized BP neural network.

be adjusted in further studies to analyze sensitivity. The MPGA–BP optimization loop is then repeated to further enhance solution accuracy and numerical stability under the simulated training distribution.

In summary, MPGA is used to optimize the initial weights and biases $\Theta^{(0)}$, while BP training with the Adam optimizer updates the network parameters during learning. Bayesian optimization is applied to tune the training hyperparameters. The network architecture (number of layers and neurons) is fixed in this study.

4. Experimental verification and simulation analysis

To evaluate the numerical performance of the proposed MPGA–BP solver for IK, simulations were conducted in MATLAB 2023 under consistent settings. The evaluation focuses on accuracy, convergence, and computational efficiency.

A total of 50,000 reachable workspace points were generated using MC sampling, and 5,000 samples were randomly selected for learning and evaluation. For each sample, the end-effector pose was computed via forward kinematics in the MATLAB Robotics Toolbox, with the corresponding six joint angles as outputs.

To prevent information leakage, the samples were randomly split into mutually exclusive subsets: 80% training, 10% validation, and 10% testing.

Inputs and outputs were normalized to improve numerical stability during training.

Figure 11 shows the RMSE (in radians) during network training. The error decreases rapidly within the first 200 epochs and reaches its minimum validation value around epoch 729. After that, the RMSE stabilizes at the order of 10^{-7} without significant oscillations, indicating stable convergence under the simulated training/validation data.

Figure 12 compares the best and worst fitness values of MPGA–BP and SPGA over 1000 generations using the same population size and fitness function. MPGA–BP achieves faster fitness improvement in the early generations and converges more smoothly than SPGA. In contrast, SPGA exhibits larger fluctuations and a lower final fitness level. The reduced best–worst fitness gap suggests a more stable evolutionary process under the same settings.

Solution accuracy is evaluated using 100 independent test samples generated within the sampled workspace. In figure 13, MPGA–BP keeps the position error below 0.0025 mm, with a mean of approximately 0.0015 mm. In comparison, SPGA exhibits larger error fluctuations with peak values of about 0.018 mm, while the standalone BP neural network shows moderate variability. In figure 14, MPGA–BP maintains attitude errors within ± 0.004 rad with relatively smooth variations, whereas SPGA shows abrupt jumps and irregular oscillations. Overall, MPGA–BP yields lower errors and smoother error profiles within the sampled workspace.

Table 4 summarizes the statistical analysis of the 100 test samples. MPGA–BP achieves mean errors of 2.4×10^{-4} mm in position and 1.2×10^{-4} rad in attitude, about one order of magnitude lower than SPGA and lower than standalone BP. These results reflect interpolation accuracy within the simulated workspace and do not account for potential errors due to model uncertainty, actuator limitations, or sensor noise in real-world robotic systems.

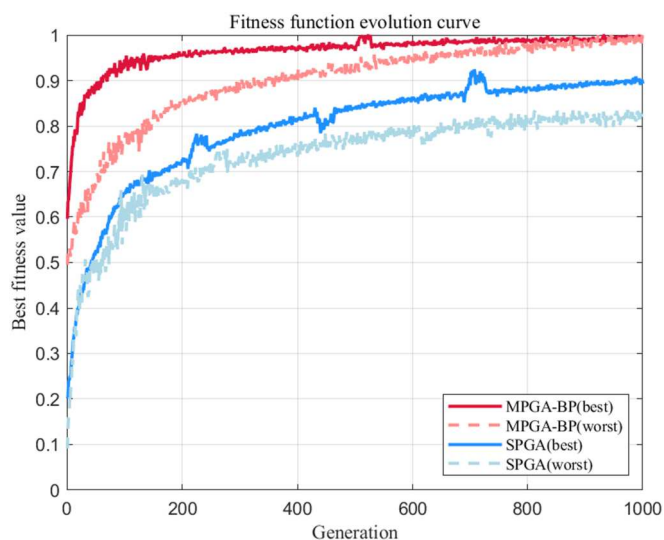


Figure 12. Evolution of the best and worst fitness values of MPGA-BP and SPGA over 1000 generations.

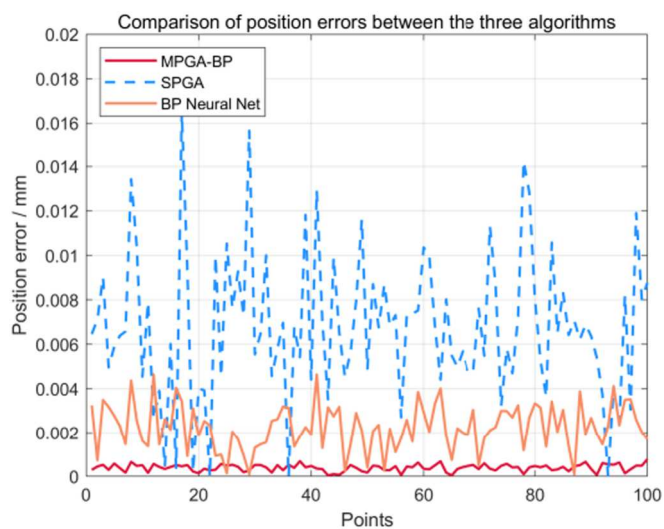


Figure 13. Position error comparison of MPGA-BP, SPGA, and BP neural network on 100 test samples.

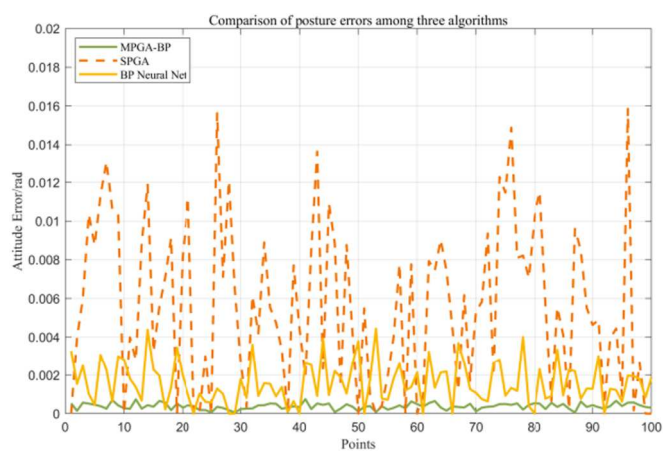


Figure 14. Attitude error comparison of MPGA-BP, SPGA, and BP neural network on 100 test samples.

Table 4. Statistical comparison of test data.

Methodologies	Average position error/mm	Average attitude error/rad
SPGA	4.8×10^{-3}	3.5×10^{-3}
BP	2.4×10^{-3}	2.2×10^{-3}
MPGA-BP	2.4×10^{-4}	1.2×10^{-4}

Table 5. IK solution results (unit: rad for joint angles, s for computation time).

Batch number	Methodologies	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6	t
1	(math.) analysis	−0.06800	−0.50000	1.48300	0.49300	0.78900	2.17300	0.480
	BP	−0.07020	−0.50450	1.48570	0.49510	0.79380	2.17050	0.003
	MPGA	−0.06780	−0.50210	1.48400	0.49320	0.79080	2.17180	0.012
	MPGA-BP	−0.06705	−0.50200	1.48365	0.49310	0.78940	2.17210	0.002
2	(math.) analysis	−0.07740	−0.48500	1.40900	0.33500	1.58000	−0.07750	0.430
	BP	−0.07610	−0.48180	1.40600	0.33690	1.58560	−0.07460	0.002
	MPGA	−0.07720	−0.48380	1.40750	0.33510	1.57710	−0.07770	0.014
	MPGA-BP	−0.07720	−0.48410	1.40795	0.33530	1.57760	−0.07790	0.002
3	(math.) analysis	0	−0.40900	1.11200	−0.03600	1.57910	0	0.320
	BP	−0.00100	−0.40600	1.11410	−0.03120	1.58350	0.00500	0.003
	MPGA	−0.00210	−0.40800	1.11210	−0.03420	1.58100	0.00320	0.013
	MPGA-BP	−0.00110	−0.40910	1.11245	−0.03510	1.58040	0.00310	0.002

All samples were generated from an ideal forward-kinematics model without sensor noise or model uncertainty; therefore, the reported errors mainly reflect interpolation accuracy under idealized conditions. In practice, encoder noise, calibration errors, and disturbances may degrade IK accuracy, and numerical stability under perturbations will be evaluated in future work.

To further examine IK performance for individual robot configurations, table 5 compares the solutions obtained using analytical IK (reference solution), BP, MPGA, and MPGA-BP. It should be noted that analytical IK solutions are not unique for general 6-DOF manipulators; therefore, the analytical results are used as reference solutions rather than absolute ground truth. BP achieves the fastest computation time (2–3 ms) but shows lower solution accuracy. MPGA improves accuracy at the cost of higher computation time. MPGA-BP provides accuracy close to the analytical solution while maintaining millisecond-level efficiency, suggesting potential for efficient real-time IK computation under simulated conditions.

Overall, MPGA-BP shows stable performance within the sampled workspace; however, numerical stability near singularities was not quantified using metrics such as the Jacobian condition number or manipulability. Training is performed offline and incurs additional cost due to MPGA initialization, whereas inference remains at the millisecond level (table 5). Retraining is required for different manipulators because the IK mapping depends on the kinematic structure; online/adaptive learning is not addressed in the current implementation.

5. Conclusions

This study presents a data-driven IK solution based on a hybrid MPGA-BP framework. Using forward-kinematics training data, the proposed method learns a mapping from end-effector pose to joint configuration for a 6-DOF manipulator.

Rather than proposing a new optimizer, this work provides a reproducible integration of MPGA-based global initialization with Adam-based gradient training, improving numerical stability and interpolation accuracy under simulated conditions.

Under the sampled workspace, MPGA-BP achieves an RMSE on the order of 10^{-7} and mean position/orientation errors of approximately 10^{-4} mm and 10^{-4} rad, respectively, with more stable convergence than SPGA and standalone BP under identical settings.

All experiments were conducted under idealized simulation assumptions; thus, the reported accuracy reflects interpolation performance rather than numerical stability under sensor noise, calibration errors, or model mismatch. Hardware-in-the-loop and closed-loop validation will be investigated in future work.

Although this study focuses on the RM65-B manipulator, the framework is transferable to other robots given appropriate training data. Future work will prioritize experimental validation, numerical stability testing under perturbations, and integration with trajectory planning and control.

Data availability statement

All data that support the findings of this study are included within the article (and any supplementary files).

Author contributions

Rong Yao

Formal analysis (lead), Methodology (lead), Writing – original draft (lead)

Jinsong Wang  0009-0004-8829-8039

Project administration (supporting), Supervision (equal), Writing – review & editing (equal)

YuHang Wei

Conceptualization (supporting), Software (equal)

Siqi Huang

Data curation (supporting), Visualization (supporting)

Yu Zhao

Investigation (supporting)

References

- [1] Spong M W, Hutchinson S and Vidyasagar M 2020 *Robot Modeling and Control* (Wiley) (<https://doi.org/10.1109/MCS.2021.3122271>)
- [2] Kucuk S and Bingul Z 2006 Robot kinematics: industrial robotics—forward and inverse kinematics *Industrial Robotics: Theory, Modelling and Control* (InTechOpen) 117–48
- [3] Xiao F, Li G, Jiang D, Xie Y, Yun J, Liu Y, Huang L and Fang Z 2021 An effective and unified method to derive the inverse kinematics formulas of general six-DOF manipulators with simple geometry *Mech. Mach. Theory* **159** 104265
- [4] Paul R P, Shimano B and Mayer G 1981 Differential kinematics control equations for simple manipulators *IEEE Trans. Syst. Man Cybern.* **11** 456–60
- [5] Siciliano B and Khatib O 2008 *Springer Handbook of Robotics* 2nd ed. (Springer Science & Business Media)
- [6] Zaplana H, Hadfield I H and Lasenby J 2022 Closed-form solutions for the inverse kinematics of serial robots using conformal geometric algebra *Mech. Mach. Theory* **173** 104835
- [7] Dong X, Yu J, Chen B and Zong G 2012 Geometric approach for kinematic analysis of a class of 2-DOF rotational parallel manipulators *Chin. J. Mech. Eng.* **25** 241–7
- [8] Andrieiev Y, Breslavsky D, Shabanov H, Naumenko K and Altenbach H 2025 Solution to the inverse problem of the angular manipulator kinematics with six degrees of freedom *Appl. Sci.* **15** 2840
- [9] Schiesser W E 1991 *The Numerical Method of Lines* (Academic)
- [10] Erleben K and Andrews S 2019 Solving inverse kinematics using exact Hessian matrices *Comput. Graph.* **78** 1–11
- [11] Xie S, Sun L, Wang Z and Chen G 2022 A speedup method for solving the inverse kinematics problem of robotic manipulators *Int. J. Adv. Robot. Syst.* **19** 17298806221104602
- [12] Gonçalves F, Ribeiro T, Ribeiro A F, Lopes G and Flores P 2022 A recursive algorithm for the forward kinematic analysis of robotic systems using Euler angles *Robotics* **11** 15
- [13] Andrieiev Y, Breslavsky D, Pashchenko S and Tatarinova O 2020 Development the algorithms of anthropomorphic robot's motion control by use of AI algorithms *In Proceedings of the (IEEE KhPI Week on Advanced Technology (KhPI Week))* **82–5**
- [14] Gao R H 2020 Inverse kinematics solution of robotics based on neural network algorithms *J. Ambient Intell. Humaniz. Comput* **11** 6199–209
- [15] Luo S, Chu D, Li Q and He Y 2022 Inverse kinematics solution of 6-DOF manipulator based on multi-objective full-parameter optimization PSO algorithm *Front. Neurorobot.* **16** 791796
- [16] Yang Y G et al 2007 A new solution for inverse kinematics of 7-DOF manipulator based on neural network *Proc. of the IEEE Int. Conf. on Automation and Logistics (ICAL) (Jinan, China)* 1958–62
- [17] Zhen S, Li J and Tian Y-T 2015 Genetic algorithm for solving the inverse kinematics problem for general 6R robots *Lect. Notes Electr. Eng.* **338** 154–64
- [18] Han X G, Song X H and Yin M 2017 Solution of inverse kinematics and welding trajectory error analysis for 6R welding robot *Trans. Chin. Soc. Agric. Mach.* **48** 384–90
- [19] Shi J P et al 2024 An improved artificial bee colony algorithm for inverse kinematics of redundant manipulators *J. Mech. Eng.* **60** 60–70
- [20] Li W B, Cao B and Zhang Y B 2016 Inverse kinematics analysis of anti-bomb robot based on BP neural network *Comput. Meas. Control* **24** 207–11
- [21] Lin Y, Zhao H and Ding H 2017 Inverse kinematics solution of general robot based on multi-population genetic algorithm *J. Mech. Eng.* **53** 1–8

- [22] Roth B, Rastegar J and Scheinman V 1973 On the design of computer controlled manipulators *On the Theory and Practice of Robots and Manipulators, Vol. 1, First CISM-IFTOMM Symp.* **93–113**
- [23] Zhang Z B, Zhang L S and Zheng Y L 2003 *Bionics in Computational Intelligence: Theory and Algorithms* (Science Press) (<https://doi.org/10.1155/2016/5720163>)
- [24] Zhou M and Sun S D 1999 *Principles and Applications of Genetic Algorithm* (National Defense Industry Press)
- [25] Almusawi A R J, Dülger L C and Kapucu S 2016 A new artificial neural network approach in solving inverse kinematics of robotic arm (Denso VP6242) *Comput. Intell. Neurosci.* **2016** 5720163